KaraoKry — Functional Spec

Oct 28, 2026 · @Carlos Noyes

1. Conversation summary and everything Carlos asked for

KaraoKry is a free, hobby party app for Carlos and his friends. A phone plays a Spotify song into the singer's Bluetooth headphones while the screen shows the lyrics of a different song. The singer sings those wrong words to the tune, and everyone else tries to guess the real song and artist. This document is the only input the coding agent will get, so everything Carlos said is recorded here.

The idea, as Carlos described it

- A lightweight phone app he can vibe-code. It is for playing with friends, not for sale.
- The singer listens to a song through Bluetooth headphones. The room cannot hear the music, only the singer.
- The screen shows lyrics that belong to a different song. The singer sings those wrong words to the tune they hear.
- The others guess the real song from the tune, rhythm and style. It plays like karaoke with a twist, and a bit like the party game Catch Phrase.
- He did not want to buy, store or host MP3s. All music and song data should come from Spotify.
- Everyone logs in with Spotify through the app, picks a playlist made for the game, and the app shuffles it. Simple controls let the singer pause, skip and replay.
- There is no scoring and there are no turns. Someone sings, then passes the phone. A competitive mode could come in a later version.

The pairing rule (Carlos's own design)

1. When a game starts, the playlist is shuffled once, immediately. That order is then locked for the whole game.
2. Songs play in that locked order. While song k plays, the screen shows the lyrics of song k−1.
3. While song 1 plays, the screen shows the lyrics of the last song (song N). The pairing wraps around.

4. Every song is therefore played once and sung once. The game stops at the end of the playlist.
5. Any song with no lyrics available is removed from the game completely, before play starts. It is neither played nor sung.
6. The playlist order and upcoming songs are never shown. Nobody should be able to see what is coming.

Carlos's answers to the design questionnaire

Topic	Carlos's decision
Platform	Build for Android first. iPhone can come later.
Spotify Premium	Carlos has Premium.
Who logs in	Ideally each friend installs the app and logs in with their own Spotify on their own phone. See section 3 for the 5-account limit this runs into.
Hosting	Carlos has a GitHub account. He asked if friends could get the app from the app store if it is hosted on GitHub Pages. Answer: no, they install it from Chrome instead (section 5).
Headphones	The phone's owner hands over their own headphones, already paired and connected. The app assumes audio output is already set up and never manages Bluetooth.
Playlist picker	Show all of the user's playlists. Carlos asked whether users could search Spotify for any playlist, such as a Disney playlist. Spotify's rules block this for hobby apps (section 3), so the app searches the user's own playlists and explains how to copy a public one.
Game length	No length setting. It is an activity like karaoke, not a game. It keeps going until the group stops or the playlist runs out.
Songs-ready count	After the lyrics check, show how many songs are ready, never which ones. Fewer than 3 usable songs means the app asks for a bigger playlist.

Topic	Carlos's decision
Auto-dropped tracks	Drop podcast episodes, local files, duplicates, instrumentals and tracks under 60 seconds. Yes to all.
Between rounds	A "pass the phone" screen that hides everything until the next singer taps "I'm ready".
Countdown	A 3-2-1 countdown before the music starts, with the first lyric lines visible during it.
Song length	Play the full song. The singer can jump forward or back at will, for example to sing the chorus again, using a YouTube-style scrub bar.
Instrumental parts	Instrumental stretches longer than 15 seconds are shown in a different colour on the scrub bar, so the singer can skip them.
Lyrics movement	The singer scrolls the lyrics by hand. They can sing any part at any point. No automatic scrolling or timing.
Lyrics size	Lyrics fit the page at a comfortable size by default. Two-finger pinch zooms the text in or out so more or less fits.
Controls	Play/pause, skip song, and easy jumping to any part of the song.
Skip safety	Skip needs a long press so it is not hit by accident.
What the singer sees	The singer may see both titles: the song playing and the song whose lyrics they are singing.
What guessers guess	The title and the band or artist of the song being played. Not the song the lyrics come from.
Ending a round	The singer presses "Got It!" when someone guesses. Add a "Go Back" button in case it was pressed by accident.
Reveal	A reveal screen shows the played song's title, artist and album art, so the singer can hold it up to the room.

Topic	Carlos's decision
Progress	Show "Song 7 of 26" style progress. Numbers only, never titles.
Lock screen leak	Spotify may show the song name on the lock screen or in notifications. Carlos is fine with that.
End of game	A simple "Game over" screen. No recap.
Resume	If the app closes mid-game, it resumes with the same locked order.
Screen sleep	Keep the screen awake during play.
Play again	Offer both: reshuffle the same playlist, or pick another playlist.
Content	Show explicit and non-English songs as they are. No filtering.
Name	KaraoKry.
Look	Dark neon karaoke bar. Standard fonts and colours are fine.
Flourishes	Confetti on "Got It!" plus vibration on Android. No sound effects, because they would play in the singer's headphones.
How to play	A short, skippable how-to-play screen for first-timers.
Icon	A custom KaraoKry icon for the home screen.
Orientation	Portrait only.
Build approach	Not answered. Default: a rough playable version first, then polish. Keep the code tidy enough to add scoring or teams later.

2. Product overview

KaraoKry is an installable web app (a PWA) for Android Chrome. It acts as a remote control for the Spotify app on the same phone and adds a lyrics screen on top. There is no server, no database and no music hosting.

One game, start to finish

1. The host opens KaraoKry and logs in with Spotify once.
2. The host picks one of their own playlists.
3. KaraoKry reads the songs, drops unusable ones, looks up lyrics for each, and drops songs without lyrics.
4. It shuffles the remaining songs once and locks that order. It shows only "24 songs ready".
5. Each round, a singer takes the phone and headphones and taps "I'm ready". A 3-2-1 countdown runs, then the song plays in their headphones.
6. The screen shows the wrong lyrics. The singer sings them to the tune, scrolling and scrubbing as they like.
7. When someone guesses, the singer taps "Got It!". The music pauses and a reveal screen shows the real song for the room.
8. The phone goes to the next singer. This repeats until the group stops or every song has been played.

Worked example of the wrap-around pairing

A playlist of five usable songs is shuffled into the locked order A, B, C, D, E.

Round	Song played (heard in headphones, guessed by the room)	Lyrics shown (sung by the singer)
1	A	E
2	B	A
3	C	B
4	D	C
5	E	D

In code terms: with the locked list `order[0..N-1]`, round `i` (0-based) plays `order[i]` and shows the lyrics of `order[(i - 1 + N) % N]`.

Glossary

Term	Meaning
Host	The person whose phone, Spotify account and headphones are used for a game.
Singer	Whoever holds the phone and wears the headphones this round.
Guessers	Everyone else in the room. They hear only the singer.
Played song	The song in the singer's headphones. The room guesses its title and artist.
Lyrics song	The song whose words appear on screen. The singer sings these words.
Locked order	The shuffled list of usable songs, fixed for the whole game.
Round	One played song, from "I'm ready" to the reveal.
Instrumental stretch	A part of the played song with no vocals for more than 15 seconds.
Spotify Connect device	The Spotify app on the phone, which KaraoKry controls through Spotify's Web API.

3. Hard platform constraints

Spotify tightened its rules for hobby apps in 2026, and several of Carlos's wishes run into them. The agent must design around every row below. Do not rely on Spotify tutorials or libraries written before February 2026: many use removed endpoints and old field names.

Spotify Development Mode (verified October 2026)

Rule	Effect on KaraoKry
Every new Spotify developer app runs in Development Mode. The higher tier (Extended Quota) is only for organisations with about 250,000 monthly users.	KaraoKry will always be a Development Mode app. Design for its limits permanently.
The app owner (Carlos) must keep an active Premium subscription, or the app stops working (migration guide).	If Carlos's Premium lapses, KaraoKry stops for everyone using his Client ID.
At most 5 Spotify accounts can use one app. Each must be added by hand, by email, in the dashboard's User Management page (Techzine).	Carlos plus up to four friends, or five friends, can log in. A sixth person is refused by Spotify. The app must explain this clearly (section 6).
Playback control needs Premium for the user who logs in.	Only friends with Premium can host a game on their own phone. Friends without Premium can still sing on someone else's phone.
Playlist contents are only returned for playlists the user owns or collaborates on. For any other playlist, only its name and cover come back (migration guide).	Searching Spotify for a public playlist such as "Disney hits" and playing it is not possible. The user must copy those songs into a playlist they own. Spotify-made playlists (Discover Weekly and similar) also cannot be used.
`GET /playlists/{id}/tracks` was replaced by `GET /playlists/{id}/items`. In responses, `tracks` became `items` and each entry's `track` became `item`.	Parse playlist data defensively. Accept both old and new field names.
Batch endpoints such as `GET /tracks?ids=` were removed. Track `popularity`, `available_markets` and `linked_from` were removed. `GET /me` no longer returns `product`, `email` or `country`.	KaraoKry cannot check for Premium up front. It detects missing Premium from playback errors instead. Everything needed must come from the playlist items response.

Rule	Effect on KaraoKry
Refresh tokens now expire 6 months after the user first logged in. Refreshing does not extend that. An expired token returns `invalid_grant` (Spotify blog).	On `invalid_grant`, discard tokens and send the user back to the login screen. Never retry the refresh.
Since July 2026, request quota is counted per developer account, and over-quota replies are HTTP 429 with reason `QUOTA_EXCEEDED` (July 2026 changelog).	Keep Web API calls few. Never poll the Web API for playback position. Honour `Retry-After` on every 429.
Redirect URIs must be HTTPS, except loopback addresses such as `http://127.0.0.1`. `localhost` is not accepted.	Production uses the GitHub Pages HTTPS URL. Local testing uses `http://127.0.0.1:<port>/`.

Playback: KaraoKry plays the music itself

Spotify's Web Playback SDK turns a web page into a Spotify Connect speaker that streams music. It supports mobile browsers, including Chrome on Android (Web Playback SDK). Audio comes out of the phone's current output, so it goes to the connected Bluetooth headphones.

This is the primary playback engine. It gives KaraoKry exact playback position and end-of-song events without any network polling. It also means the Spotify app does not need to be open.

Known limits that the design must handle:

- Mobile browsers only start audio after a user tap. The first `play` must happen inside a tap handler. The "I'm ready" button provides that tap.
- Browsers may stop audio when the page goes to the background or the screen turns off. KaraoKry keeps the screen awake and expects to stay in the foreground during a round.
- Some Android phones fail to start the SDK because of DRM settings (Chrome's "Protected content" setting, or a phone that dropped to a lower Widevine level). This shows up as an SDK initialisation or playback error.
- If the SDK fails, KaraoKry falls back to remote-controlling the Spotify app through Spotify Connect (section 9). The fallback works but needs the Spotify app installed and gives less precise position updates.

Lyrics: LRCLIB

Spotify does not give lyrics to developers. KaraoKry uses LRCLIB, a free, open lyrics database.

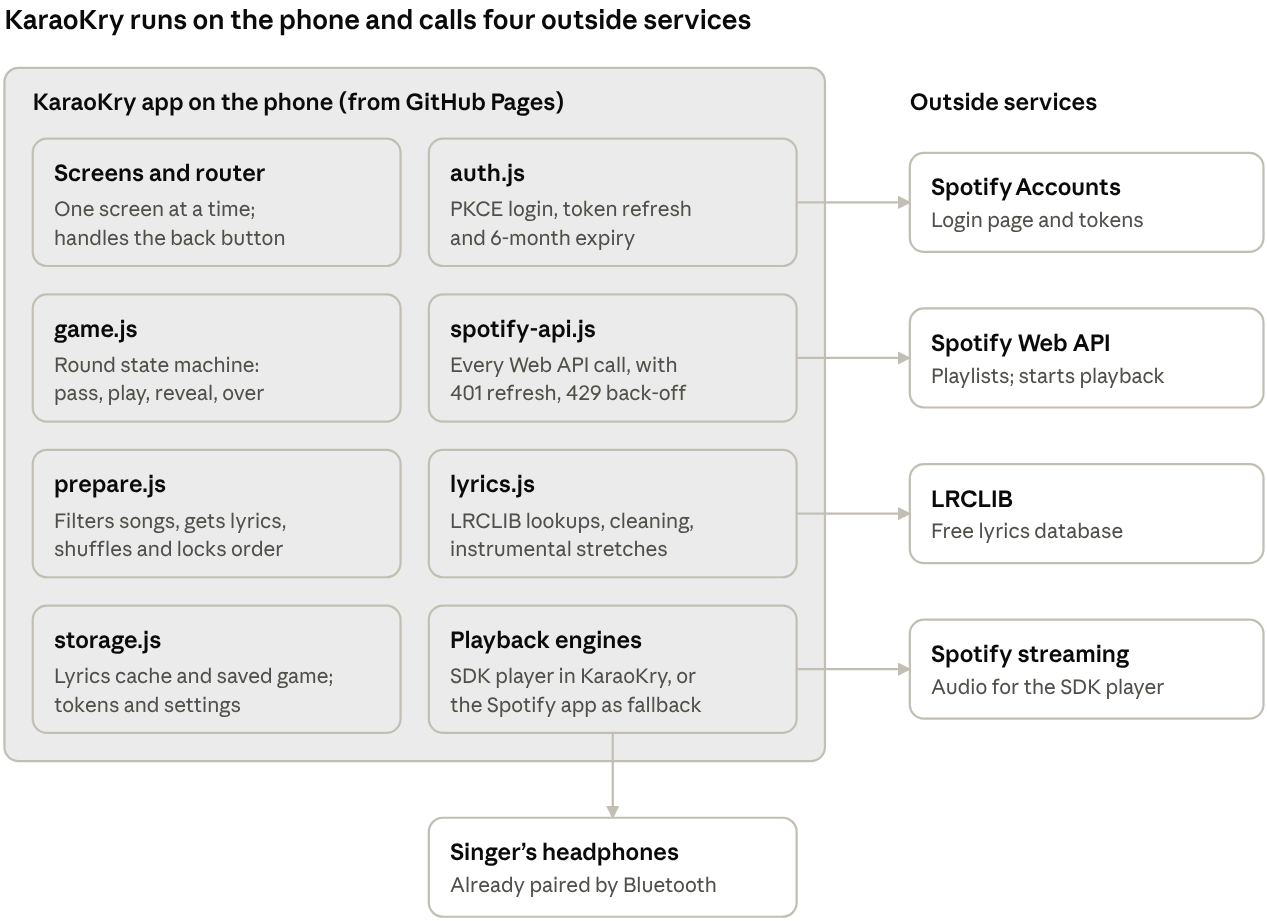
- No API key, no registration and no rate limiting.
- Browsers cannot set a `User-Agent` header, so LRCLIB accepts a `Lrclib-Client` header instead. Send `Lrclib-Client: KaraoKry v1 (<GitHub repo URL>)`.
- `GET /api/get` matches by track name, artist, album and duration. It is fast and preferred. `GET /api/search` is a slower fallback.
- Each record has `plainLyrics`, `syncedLyrics` (time-stamped LRC lines) and an `instrumental` flag.
- Coverage is good for popular music but not complete. Missing lyrics are expected and handled by dropping the song.

Other platform facts

- A web app on GitHub Pages cannot be listed in the Google Play Store as is. Friends install it from Chrome with "Install app" or "Add to Home screen", and it then opens full-screen like a normal app.
- Spotify, and Android's media notification, may show the playing song's name on the lock screen or in the notification shade. Carlos accepted this.

4. Architecture and tech stack

KaraoKry is a static, single-page Progressive Web App with no backend. Every file is served from GitHub Pages, and the phone talks directly to Spotify and LRCLIB.



architecture · 8 modules, 4 outside services

Everything inside the box is static files from GitHub Pages; there is no KaraoKry server. Only the right-hand modules talk to the outside world.

Technology choices

Choice	Decision	Why
Language	Plain HTML, CSS and modern JavaScript (ES modules)	Easy to vibe-code and read. No compile step.
Framework	None	The app has about ten screens and little shared state. A framework adds setup without much benefit.
Build step	None. Files are served exactly as written.	GitHub Pages can serve the repository directly.

Choice	Decision	Why
Third-party code	Only Spotify's SDK script, `https://sdk.scdn.co/spotify-player.js`, loaded at runtime	Fewer things to break. Confetti, haptics and storage helpers are small hand-written modules.
Auth	OAuth 2.0 Authorization Code with PKCE, entirely in the browser	Spotify's recommended flow for apps with no server. No client secret is ever needed or stored.
Storage	`localStorage` for tokens and small settings. IndexedDB for the lyrics cache and the saved game.	A saved game with lyrics can exceed what is comfortable in `localStorage`.
Offline	A service worker caches the app shell only	Lets the installed app open instantly. Spotify and LRCLIB still need a connection.
Target browser	Chrome for Android, current version	Carlos asked for Android first. Desktop Chrome is used for development. iPhone Safari is out of scope for v1 but nothing should block it later.

Hosting and URLs

- Repository name: `karaokry`. Published with GitHub Pages from the `main` branch root.
- Production URL: `https://<github-username>.github.io/karaokry/`. Every path in the app must be relative, because the site lives under `/karaokry/`, not at the domain root.
- The OAuth redirect comes back to the same page: `https://<github-username>.github.io/karaokry/`. The app reads `?code=` and `?state=` on load, then removes them from the address bar with `history.replaceState`.
- Local development: serve the folder with any static server on `http://127.0.0.1:5500/`. Register that as a second redirect URI.

File structure

```
karaokry/
  index.html                  single page; each screen is a <section> shown or
hidden by the router
  manifest.webmanifest        PWA manifest (name, icons, colours, portrait)
  sw.js                       service worker: app-shell cache only
  config.js                   SPOTIFY_CLIENT_ID default and app version
  README.md                   setup steps from section 5, for Carlos
  css/
    app.css                   design tokens and all styles
  icons/
    icon.svg, icon-192.png, icon-512.png, icon-maskable-512.png
  js/
    main.js                   boot: register service worker, finish login,
resume game, start router
    router.js                 shows one screen at a time; manages history for
the Android back button
    auth.js                   PKCE login, token storage, refresh, logout
    spotify-api.js            fetch wrapper: bearer token, 401 refresh, 429
back-off, error mapping
    playlists.js              list playlists, read playlist items
    prepare.js                filter tracks, fetch lyrics, shuffle, build the
locked order
    lyrics.js                 LRCLIB client, cleaning, LRC parsing,
instrumental gap detection
    playback/
      engine.js               the PlaybackEngine interface both engines
implement
      sdk-engine.js           plays inside KaraoKry with the Web Playback SDK
(primary)
      connect-engine.js       remote-controls the Spotify app (fallback)
    game.js                   game state machine, rounds, persistence
    storage.js                tiny IndexedDB key-value wrapper
    wake-lock.js              Screen Wake Lock handling
    ui/
      screens/                one module per screen (section 7)
      lyrics-view.js          lyrics rendering, scrolling, pinch zoom
      scrub-bar.js            progress bar, dragging, instrumental shading
```

```
      long-press.js          press-and-hold helper with a progress ring
      confetti.js            canvas confetti burst
      haptics.js             navigator.vibrate wrapper
      toast.js               short non-blocking messages
```

Rules for the code

- Keep each module small and single-purpose, so Carlos or another AI can change one part without reading everything.
- All Spotify calls go through `spotify-api.js`. All LRCLIB calls go through `lyrics.js`. No other module calls `fetch` directly.
- The two playback engines expose the same interface (section 9), so the rest of the app never knows which one is active.
- Game logic in `game.js` must not touch the DOM. Screens read state from it and call its actions. This keeps a future scoring or teams mode easy to add.
- Never log tokens. Never put tokens or personal data in URLs, except the one-time `code` that Spotify itself returns.

5. Setup and distribution

Carlos must do three one-time setup jobs himself, because they need his own accounts. The agent cannot do them, but must write them into `README.md` exactly as below and tell Carlos when each is needed.

A. Publish the site on GitHub Pages (Carlos, about 5 minutes)

1. Create a public GitHub repository named `karaokry`. Free GitHub accounts can only use Pages with public repositories.
2. Push the project files to the `main` branch.
3. In the repository, open Settings, then Pages. Under "Build and deployment", choose "Deploy from a branch", branch `main`, folder `/ (root)`, and save.
4. After a minute or two the site is live at `https://<github-username>.github.io/karaokry/`.

B. Register the Spotify app (Carlos, about 5 minutes)

1. Go to the Spotify Developer Dashboard at `developer.spotify.com/dashboard` and log in with Carlos's Premium account.

2. Create an app. Name: `KaraoKry`. Description: a short line such as "Party lyrics game for friends".
3. Redirect URIs, add both, exactly:
   - `https://<github-username>.github.io/karaokry/`
   - `http://127.0.0.1:5500/` (for local testing)
4. Under the APIs used, tick Web API and Web Playback SDK.
5. Accept the terms and save. Copy the Client ID.
6. Paste the Client ID into `config.js` as `SPOTIFY_CLIENT_ID`, commit and push. A Client ID is not a secret and is safe in a public repository. There is no client secret in this app.

C. Let friends log in (Carlos, 1 minute per friend)

1. In the dashboard, open the KaraoKry app, then Settings, then User Management.
2. Add each friend's full name and the email address of their Spotify account.
3. The limit is 5 accounts in total. Carlos counts toward it if he logs in himself.
4. A friend can only host a game on their own phone if their Spotify is Premium.

D. Installing on an Android phone (each user)

1. Open the site URL in Chrome.
2. Tap KaraoKry's own "Install KaraoKry" button on the welcome screen. It appears when Chrome fires `beforeinstallprompt`.
3. If that button is missing, use Chrome's menu (three dots) and choose "Install app" or "Add to Home screen".
4. KaraoKry then has its own icon and opens full-screen in portrait, like any app.
5. Updates arrive automatically. When a new version is deployed, the app shows "New version ready, tap to refresh".

App stores

KaraoKry is not published to the Google Play Store in v1. Carlos asked about this. A web app on GitHub Pages is installed from Chrome instead, which looks and behaves like a store app.

A Play Store listing would mean wrapping the site with a Trusted Web Activity tool (such as Bubblewrap), a paid Google Play developer account, and Google's review. It would not help anyway, because Spotify's 5-account limit still applies. Leave this as a future idea

only.

Friends beyond the 5-account limit

Settings includes an advanced field, "Use my own Spotify app (Client ID)". A friend with Premium can do step B themselves with the same redirect URI, then paste their own Client ID there. That friend's app then has its own 5-account allowance. The field is hidden under an "Advanced" toggle and links to the README steps.

6. Authentication and accounts

KaraoKry logs in with Spotify's Authorization Code flow with PKCE, done entirely in the browser. The user logs in once per phone and stays logged in until the 6-month refresh token expires or they log out.

Scopes

Request exactly these scopes, space-separated:

Scope	Needed for
`streaming`	Web Playback SDK playback inside KaraoKry
`user-read-email`	Required by the Web Playback SDK
`user-read-private`	Required by the Web Playback SDK, and `GET /me`
`playlist-read-private`	Listing and reading the user's private playlists
`playlist-read-collaborative`	Listing and reading collaborative playlists
`user-read-playback-state`	Device list and playback state (fallback engine)
`user-modify-playback-state`	Starting, pausing and seeking playback

Login steps

1. Generate a `code_verifier`: 64 random characters from `A-Z a-z 0-9 - . _ ~`, using `crypto.getRandomValues`.
2. Compute `code_challenge` = base64url of the SHA-256 of the verifier, with no padding. Use `crypto.subtle.digest`.

3. Generate a random `state` of 16 bytes, base64url-encoded.
4. Save the verifier, state and the Client ID in use to `localStorage` under `kk.pkce`. Use `localStorage`, not `sessionStorage`, because on Android the login page can open in a different window from the installed app.
5. Navigate to `https://accounts.spotify.com/authorize` with `response_type=code`, `client_id`, `scope`, `redirect_uri`, `state`, `code_challenge_method=S256` and `code_challenge`.
6. On return, `main.js` sees `?code=` and `?state=`. If `state` does not match the saved value, show an error and offer "Try logging in again".
7. POST to `https://accounts.spotify.com/api/token` as `application/x-www-form-urlencoded` with `grant_type=authorization_code`, `code`, `redirect_uri`, `client_id` and `code_verifier`.
8. Store `access_token`, `refresh_token`, `expires_at` (now + `expires_in` seconds) and `authorized_at` in `localStorage` under `kk.auth`. Delete `kk.pkce`.
9. Remove `code` and `state` from the address bar with `history.replaceState`.
10. Call `GET /me` to confirm access and to read `id` and `display_name`. Then continue to the next screen (section 7).

If the login finishes in a normal Chrome tab instead of the installed app window, that tab shows "You're logged in. You can close this tab and go back to KaraoKry." The installed app listens for the `storage` event and on `visibilitychange` re-reads `kk.auth`, so it picks up the login when the user returns.

If Spotify returns `?error=access_denied`, go back to the welcome screen with the message "Spotify login was cancelled."

Keeping the token fresh

- Access tokens last about one hour. Before any Spotify call, refresh if `expires_at` is less than 60 seconds away.
- Only one refresh may run at a time. Other callers wait for the same promise.
- Refresh with `grant_type=refresh_token`, `refresh_token` and `client_id`. If the response contains a new `refresh_token`, replace the stored one. Keep the original `authorized_at`.
- If a Spotify call returns 401, refresh once and retry the call once.

- If the refresh returns `invalid_grant`, the 6-month limit or a revoked login has been reached. Do not retry. Clear `kk.auth`, keep the saved game, and show "Your Spotify login has expired. Please log in again." After login, offer to resume the saved game.
- The Web Playback SDK asks for tokens through its `getOAuthToken` callback. That callback must use the same refresh logic.

Not on the guest list

In Development Mode, Spotify rejects accounts that are not in the app's User Management list. KaraoKry sees this as an HTTP 403 from `GET /me` right after login.

When that happens, show the "Not on the guest list" screen:

- Title: "You're not on the guest list yet".
- Body: "Spotify only lets 5 people use a hobby app like KaraoKry. Ask the person who runs KaraoKry to add the email address of your Spotify account. Then tap Try again."
- Buttons: "Try again" (calls `GET /me` again), "Log out", and a small "Advanced: use my own Spotify app" link to the Client ID setting.

Client ID setting

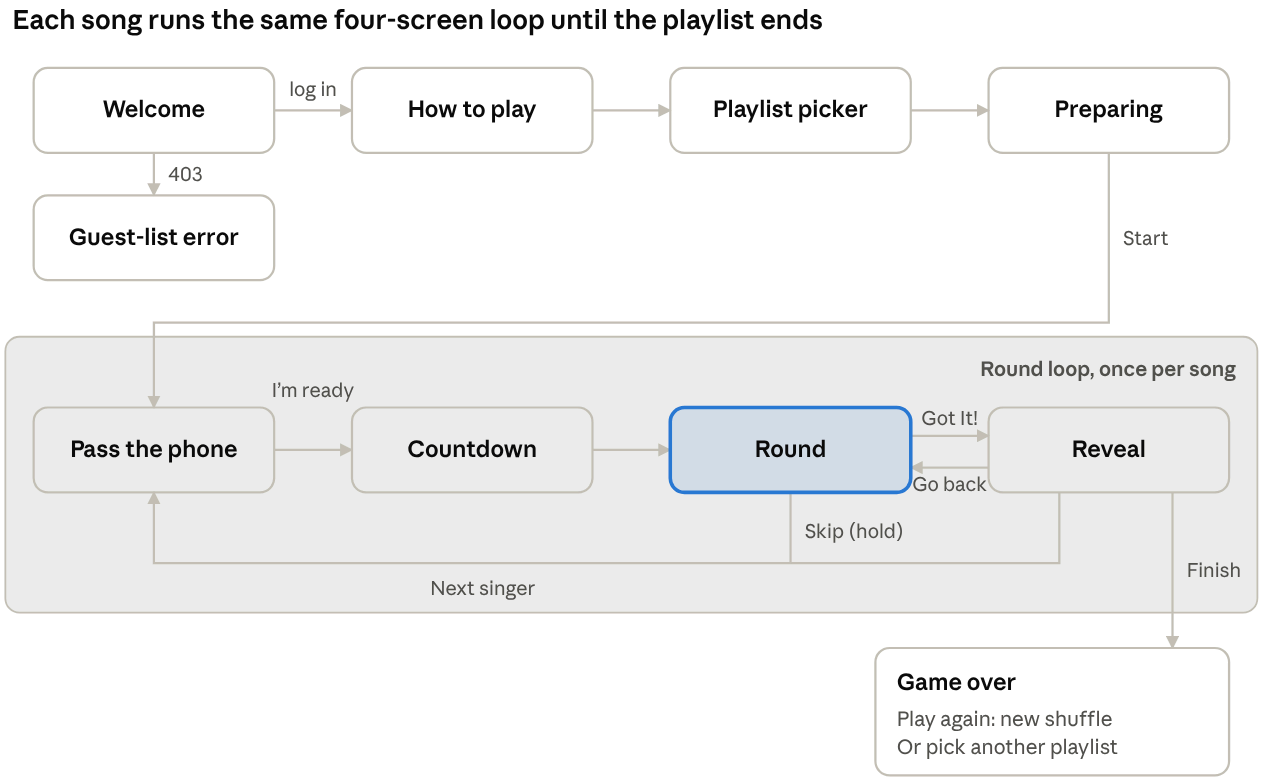
- `config.js` holds Carlos's Client ID as the default.
- Settings has an "Advanced" toggle with a text field for a custom Client ID. Validate it as 32 hexadecimal characters.
- Saving a new Client ID logs the user out and starts a fresh login with that ID. Store it in `localStorage` under `kk.clientId`.
- A "Reset to default" button clears it.

Logging out

Log out from Settings. It disconnects the SDK player, clears `kk.auth` and the PKCE data, and deletes the saved game. It keeps the lyrics cache and display settings. It returns to the welcome screen.

7. Screens and navigation

KaraoKry has 12 screens, shown one at a time inside `index.html`. The heart of the app is a loop of four screens: Pass the phone, Countdown, Round and Reveal.



screen flow · 10 screens, one loop

Not drawn: the Resume prompt opens straight into the loop; Settings and How to play open from the menu; End game jumps to Game over from Pass the phone or Round; a finished song also leads to Reveal.

Screen list

#	Screen	When it appears	What it shows	Actions
1	Welcome	First launch, or when logged out	KaraoKry logo, tagline "Sing it wrong.", one-line description	"Log in with Spotify". "Install KaraoKry" when Chrome offers it. "How to play" link.
2	Not on the guest list	`GET /me` returns 403 after login	Explanation from section 6	Try again, Log out, Advanced Client ID
3	How to play	Automatically after the first successful login, and from the menu	Three short steps, one per card (text below)	Next, Skip, and "Let's go" on the last card

#	Screen	When it appears	What it shows	Actions
4	Resume prompt	On launch when a saved game exists	"Pick up where you left off?", playlist name, "Song 7 of 24"	"Resume" and "New game". New game asks for confirmation, then deletes the saved game.
5	Playlist picker	After login, after "Pick another playlist", or with no saved game	Search box, usable playlists, followed playlists that can't be used, copy tip	Tap a usable playlist to prepare it. Settings icon. Refresh.
6	Preparing	After choosing a playlist	Progress while songs are checked, then the songs-ready result	Cancel while working. Then "Start" or "Back".
7	Pass the phone	Before every round, including the first	"Song 7 of 24", "Pass the phone to the next singer", headphones hint	"I'm ready" (big). Menu: End game, Settings, How to play.
8	Countdown	After "I'm ready"	3, 2, 1 over the round screen, with the first lyric lines visible	None. It runs for 3 seconds.
9	Round	While the singer sings	Titles, lyrics, scrub bar, controls (section 10)	Play/pause, scrub, skip (long press), Got It!, menu
10	Reveal	After "Got It!", or "Reveal" when the song ended	Album art, title, artist, and the lyrics song in smaller text	"Next singer" (or "Finish" on the last round), "Go back"
11	Game over	After the last round, or after "End game"	"Game over", playlist name, number of songs played	"Play again (new shuffle)", "Pick another playlist"

#	Screen	When it appears	What it shows	Actions
12	Settings	From the menu or the picker's settings icon	Account, lyrics size, playback mode, advanced options	See section 12

How to play text

1. "Pick a playlist you made. KaraoKry shuffles it and finds the lyrics for every song."
2. "Put on the headphones. You'll hear one song but see the words to a different one. Sing those words to the tune you hear."
3. "Everyone else guesses the song and the artist. When someone gets it, tap Got It! and pass the phone."

Record that it was seen in `localStorage` as `kk.seenHowTo = true`.

Pass the phone screen details

- Background is plain and dark. No album art, titles, colours from covers, or any hint of the coming songs.
- Copy: "Pass the phone to the next singer". Hint below: "Headphones on? Only the singer should see the screen from here."
- Progress chip at the top: "Song 7 of 24".
- The "I'm ready" tap is also the user gesture that unlocks audio for the Web Playback SDK (section 9).

Ending a game early

The menu on the Pass the phone and Round screens has "End game". It asks "End this game? You won't be able to resume it." with "End game" and "Cancel". Ending pauses playback, deletes the saved game and shows Game over.

Android back button

The router pushes a history entry for each screen and handles `popstate`, so the hardware back gesture never closes the app by surprise.

Screen	Back does
Welcome, Playlist picker	Default behaviour (leaves the app)
How to play, Settings	Closes it and returns to the previous screen
Preparing	Cancels preparation and returns to the picker
Pass the phone	Opens the End game confirmation
Countdown	Cancels the countdown and returns to Pass the phone. Music has not started.
Round	Pauses the music and opens the menu: Resume, Skip this song, End game
Reveal	Same as "Go back"
Game over	Goes to the playlist picker

8. Game preparation

Preparing a game turns a playlist into a locked, shuffled list of at least 3 songs that all have lyrics. All of this happens before the first round, so nothing is looked up mid-game.

8.1 Playlist picker data

1. Read the user's id from `GET /me` (already done at login).
2. Call `GET /me/playlists?limit=50` and follow each response's `next` URL until it is null.
3. For each playlist keep: `id`, `name`, the first image URL (images may be empty or null), `owner.id`, `collaborative`, and the song count from `items.total`, falling back to `tracks.total`.
4. A playlist is usable when `owner.id` equals the user's id, or `collaborative` is true.
5. Show usable playlists first, in Spotify's order. Hide playlists with a song count under 3, or show them disabled with "Too short".
6. Show non-usable playlists in a collapsed section titled "Playlists you follow (can't be used)", greyed out.

7. Tapping a non-usable playlist opens a sheet: "Spotify doesn't let hobby apps read playlists you don't own. To play this one, open it in Spotify and add its songs to a playlist you made (look for 'Add to other playlist' in the playlist's menu). It will then show up here."
8. A permanent tip under the search box says: "Want a playlist you didn't make, like a Disney mix? Copy its songs into one of your own playlists in Spotify."

The search box filters the list as the user types. Match the typed text anywhere in the playlist name, ignoring case and accents (compare `normalize('NFD')` with combining marks removed). It searches the user's own list only, because searching all of Spotify cannot lead to a playable game (section 3).

8.2 Reading the playlist

1. Call `GET /playlists/{id}/items?limit=50&additional_types=track` and follow `next` until it is null.
2. For each entry, read the track as `entry.item ?? entry.track`, and the local-file flag as `entry.is_local`.
3. If the call returns 403, or the response has no items field, show: "Spotify won't let KaraoKry read this playlist. Copy its songs into a playlist you own." Return to the picker.

8.3 Filtering tracks

Apply these filters in order. Record an internal reason code for each dropped track for debugging, but never show which songs were dropped.

Order	Drop when	Reason code
1	The entry's track is null	`missing`
2	`is_local` is true, or the URI starts with `spotify:local:`	`local`
3	`type` is not `track` (podcast episodes and others)	`not_track`
4	`is_playable` is present and false	`unplayable`
5	`duration_ms` is under 60,000	`too_short`
6	The same track id appeared earlier	`duplicate`

Order	Drop when	Reason code
7	The same cleaned title and first artist appeared earlier (another version of the same song)	`duplicate_version`
8	No usable lyrics were found, or LRCLIB marks it instrumental (8.4)	`no_lyrics`

For each kept track store: `id`, `uri`, `name`, `artists` (array of names), `album` name, `imageUrl` (the largest album image up to 640 px), `imageUrlSmall` (the smallest), `durationMs` and `explicit`.

8.4 Lyrics lookup

Look up lyrics for every track that survived filters 1 to 7.

1. Check the cache first. IndexedDB key `lyrics:<spotify track id>`. Found lyrics stay valid for 30 days, "not found" results for 7 days.
2. Try an exact match: `GET https://lrclib.net/api/get` with `track_name`, `artist_name` (the first artist), `album_name` and `duration` (track length in whole seconds).
3. On 404, clean the title (rules below) and try `GET https://lrclib.net/api/search?track_name=<cleaned title>&artist_name=<first artist>`.
4. From the search results, skip records with `instrumental: true` or with both lyric fields empty. Prefer a duration within 3 seconds of the Spotify track. Accept up to 10 seconds if the cleaned titles match exactly. Among equals, prefer records that have `syncedLyrics`.
5. If nothing qualifies, the result is "not found". Cache it and drop the track.

Every LRCLIB request sends `Lrclib-Client: KaraoKry v1 (<repo URL>)` and has a 10-second timeout. Retry once after 1 second on a network error or 5xx. Run at most 4 requests at a time. Use an `AbortController` so the Cancel button stops everything.

If 5 requests in a row fail with network errors, stop and show "Couldn't reach the lyrics service. Check your connection and try again."

Title cleaning rules, used for search and duplicate detection:

- Remove a trailing `-` part when it mentions remaster, live, edit, version, mono, stereo, mix, acoustic, demo, bonus, or "from".
- Remove brackets or parentheses containing `feat.`, `ft.` or `with`.

- Trim spaces, lower-case, and remove accents when comparing.

Lyrics cleaning rules, applied to the found text:

- Use `plainLyrics`. If it is empty, build the plain text from `syncedLyrics` by removing the `[mm:ss.xx]` time stamps.
- Normalise line endings to `\n` and trim each line.
- Remove lines that are only a section tag in square brackets, such as `[Chorus]`.
- Collapse runs of blank lines into a single blank line, which marks a verse break.
- If fewer than 4 non-empty lines remain, treat the song as having no lyrics.

8.5 Instrumental stretches

For each kept track that has `syncedLyrics`, work out where its long instrumental parts are. These are drawn on the scrub bar when that track is the song being played (section 10).

1. Parse the LRC lines into `{ t: seconds, text }` and sort by `t`.
2. Vocal lines are lines whose text is not empty and is not only symbols such as "♪" or "...".
3. A vocal line ends at the start of the next line of any kind, but no later than 7 seconds after it starts.
4. Candidate gaps are: the intro from 0 to the first vocal line, each space between one vocal line's end and the next vocal line's start, and the outro from the last vocal line's end to the track's end.
5. Keep gaps longer than 15 seconds. Store them as `instrumentalStretches: [{ start, end }]` in seconds.
6. Tracks without synced lyrics get an empty list, so their bar shows no shading.

8.6 Minimum size

At least 3 songs must remain. Otherwise show: "Only 2 songs in this playlist have lyrics we can find. Add more songs or pick another playlist." Use the real number, with a Back button.

8.7 Shuffle and lock

1. Shuffle the kept tracks once with a Fisher-Yates shuffle driven by `crypto.getRandomValues`.
2. That array is the locked order. It never changes during the game.

3. Round `i` (0-based) plays `songs[i]` and shows the lyrics of `songs[(i - 1 + N) % N]`.
4. Save the game immediately (section 12), before the first round.

8.8 Preparing screen

- While working: the playlist's cover and name, a progress bar with "Checked 38 of 112 songs", and a rotating line every 3 seconds from: "Tuning the off-key machine…", "Mixing up the lyrics…", "Hiding the answers…", "Warming up the mic…".
- When done: "24 songs ready" in large text. Underneath, small: "6 songs left out (no lyrics found, or not playable)." Never list titles.
- Buttons: "Start" (goes to Pass the phone for song 1) and "Back".
- A 100-song playlist takes roughly 10 to 30 seconds the first time and is almost instant later, thanks to the cache.

9. Playback control

KaraoKry plays music through one of two engines with an identical interface. The Web Playback SDK engine plays audio inside KaraoKry and is the default. The Spotify Connect engine remote-controls the Spotify app and is the fallback.

9.1 The PlaybackEngine interface

```
// js/playback/engine.js
// Both engines implement exactly this shape.
{
  name,                       // 'sdk' or 'connect'
  async init(),               // get a device ready; throws
EngineUnavailableError if impossible
  unlock(),                   // call synchronously inside a user tap, before
any play
  async playTrack(uri, positionMs = 0),
  async pause(),
  async resume(),
  async seek(positionMs),
  getPosition(),              // cheap local estimate: { positionMs,
durationMs, paused, uri }
  on(eventName, handler),     // 'state', 'ended', 'interrupted', 'error'
  async destroy()
}
```

The round screen only ever calls this interface. It never calls Spotify directly.

9.2 Choosing the engine

Settings has "Playback" with three options. The default is Automatic.

Option	Behaviour
Automatic (default)	Try the SDK engine. If it fails to start or reports a DRM or playback error, switch to the Connect engine and remember that choice for this phone in `localStorage` as `kk.engine = 'connect'`.
Play inside KaraoKry	SDK engine only. Errors are shown and nothing falls back.
Use the Spotify app	Connect engine only.

When Automatic switches engines, show the toast "Using the Spotify app for playback on this phone." Choosing Automatic again in Settings clears the remembered choice.

9.3 SDK engine (primary)

Set-up, done once per game when the user taps "Start" on the Preparing screen:

1. Define `window.onSpotifyWebPlaybackSDKReady`, then load `https://sdk.scdn.co/spotify-player.js` once.
2. Create `new Spotify.Player({ name: 'KaraoKry', getOAuthToken: cb => auth.getValidToken().then(cb), volume: 1.0 })`.
3. Add listeners for `ready`, `not_ready`, `initialization_error`, `authentication_error`, `account_error`, `playback_error`, `autoplay_failed` and `player_state_changed`.
4. Call `player.connect()`. On `ready`, store `device_id`. Allow 10 seconds. If `ready` never fires, treat it as an engine failure.

Each round:

1. In the "I'm ready" tap handler, call `player.activateElement()` synchronously. This satisfies the browser's rule that audio must start from a tap.
2. When the countdown ends, call `PUT https://api.spotify.com/v1/me/player/play?device_id=<id>` with body `{ "uris": ["<track uri>"], "position_ms": 0 }`.
3. If that returns 404 (device not registered yet), wait 1 second and retry, up to 3 times.
4. On the first round of a game, after playback starts, call `PUT /me/player/repeat?state=off&device_id=<id>` and `PUT /me/player/shuffle?state=false&device_id=<id>`.
5. Pause, resume and seek use the SDK's own `player.pause()`, `player.resume()` and `player.seek(ms)`. They need no Web API calls.
6. For position, call `player.getCurrentState()` every 500 ms while the round screen is visible. Between reads, move the bar smoothly with `requestAnimationFrame` when not paused.

Error mapping:

SDK event	What KaraoKry does
`account_error`	Show "Spotify Premium is needed to host a game on this phone." Return to the picker. Do not fall back, because the Connect engine also needs Premium.

SDK event	What KaraoKry does
`authentication_error`	Refresh the token once and reconnect. If it fails again, send the user to log in.
`initialization_error`, `playback_error`	Automatic mode: switch to the Connect engine and replay the current song from its last position. Otherwise show the error.
`autoplay_failed`	Show a large "Tap to start the music" button. Its tap calls `activateElement()` then `resume()`.
`not_ready`	Show "Reconnecting..." and call `connect()` again. Keep the round state.

9.4 Connect engine (fallback)

This engine needs the Spotify app installed on the phone and logged in to the same account.

1. Find a device with `GET /me/player/devices`. Prefer the active device, otherwise the one with `type` "Smartphone".
2. If there is none, show the "Wake up Spotify" helper: "KaraoKry needs the Spotify app open for a moment." A button "Open Spotify" opens the app (link to `spotify:`). The text says "Then come straight back here." While visible, poll the device list every 2 seconds for up to 60 seconds.
3. To play, call `PUT /me/player/play?device_id=<id>` with the same body as the SDK engine. If it returns 404, call `PUT /me/player` with `{ "device_ids": ["<id>"], "play": false }`, then retry the play once.
4. Set repeat off and shuffle off after the first play, as in 9.3.
5. Pause with `PUT /me/player/pause`. Resume with `PUT /me/player/play?device_id=<id>` and no body. Seek with `PUT /me/player/seek?position_ms=<ms>`.
6. For position, call `GET /me/player` every 3 seconds while a round is playing, and 300 ms after every command. Do not poll while paused. Interpolate between polls.
7. If any call returns 403 with reason `PREMIUM_REQUIRED`, show the Premium message from 9.3.

9.5 Detecting the end of a song

Both engines emit `ended` when the round's song finishes, and must stop Spotify from carrying on into another song.

- Remember the URI reported in the first state update after `playTrack`. Spotify can substitute an equivalent version of a track, so that observed URI is the one to compare against.
- The song has ended when the position reached the last 2 seconds of the track and the next state shows it paused at position 0, or shows a different track.
- If a different track starts after the song ended (Spotify's autoplay), pause it immediately.
- If a different track appears before the song neared its end, someone has taken over playback, for example from another device. Pause, emit `interrupted`, and show "Playback was interrupted. Tap play to continue." Tapping play calls `playTrack(roundUri, lastKnownPosition)`.

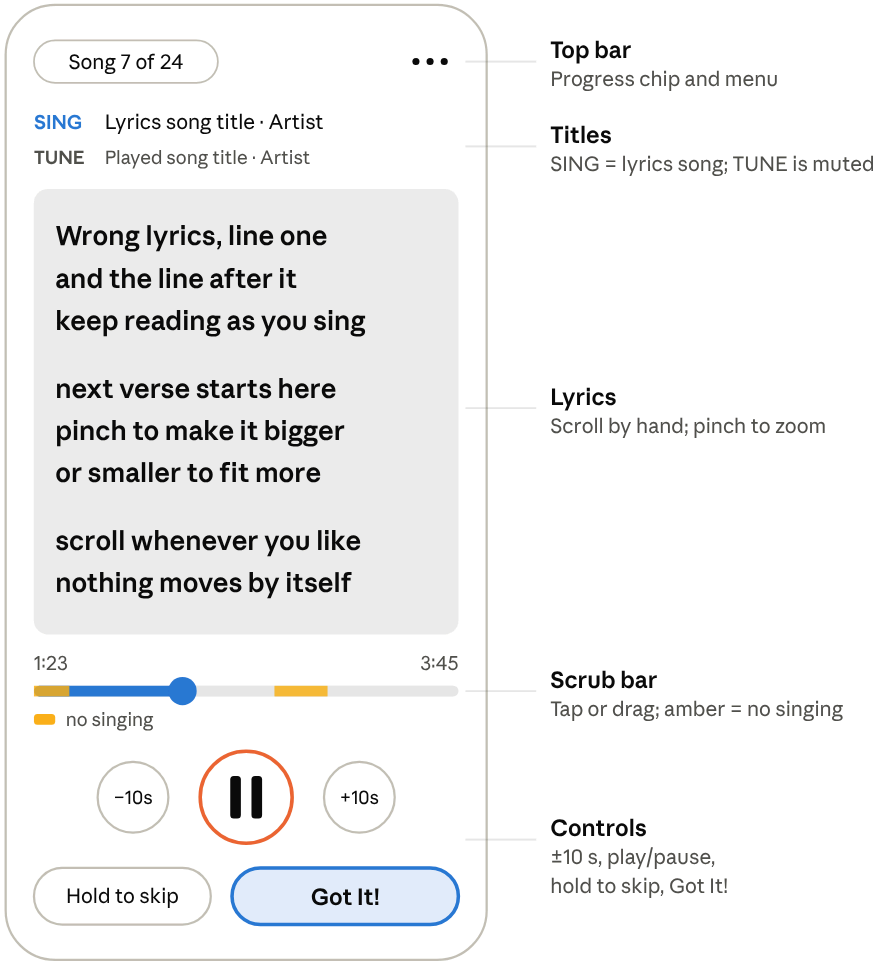
9.6 Other rules

- Volume is controlled with the phone's own volume buttons. KaraoKry has no volume control.
- KaraoKry never checks or changes the audio output. Carlos confirmed the headphones are already connected.
- When the page becomes hidden mid-round, the browser may stop audio. When it becomes visible again and playback is paused unexpectedly, show the "Tap play to continue" state.
- All Web API calls go through `spotify-api.js`. On 429, wait for `Retry-After` seconds (default 2) and try again, at most 3 times.

10. The round screen

The round screen gives the singer the wrong lyrics to scroll freely, plus full control over where they are in the song. Nothing on it moves automatically except the playback position.

Lyrics fill the middle; every control sits at the bottom



round screen layout sketch · portrait phone

The sketch shows placement only. Real colours come from section 13: pink for SING, the played part of the bar and Got It!; cyan for the play ring; amber for no-singing stretches.

10.1 Layout, top to bottom (portrait)

1. **Top bar.** Left: the "Song 7 of 24" chip. Right: a menu button (three dots) with Resume, Skip this song, End game, Settings and How to play.
2. **Titles.** Two compact lines, each truncated with an ellipsis if too long:
   - A pink "SING" label, then the lyrics song's title and artist.
   - A smaller, muted cyan "TUNE" label, then the played song's title and artist.
   - Carlos said the singer may see both. Keep the TUNE line small and low-contrast so guessers glancing at the phone are unlikely to read it. Do not show album art during the round.

3. **Lyrics area.** Fills all remaining height and scrolls vertically (10.2).
4. **Scrub bar.** Elapsed time on the left, total length on the right, and the bar between (10.3).
5. **Controls.** A row with "−10s", a large play/pause button, and "+10s". Below it, a row with "Hold to skip" on the left and a large "Got It!" button on the right (10.4).

Use `100dvh` for the screen height and respect safe-area insets, so nothing hides behind Android's gesture bar.

10.2 Lyrics area

- Show the lyrics song's cleaned plain lyrics (section 8.4). One lyric line per line; long lines wrap. A blank line becomes a verse gap of 0.8 em.
- Left-aligned, line height 1.35, with 16 px side padding. Fade the top and bottom 24 px edges with a gradient mask so it is clear there is more to scroll.
- The singer scrolls by hand with normal touch scrolling. There is no auto-scroll, no highlighting and no timing.
- Default text size: `clamp(22px, 6.5vw, 34px)`. This shows roughly 8 to 12 lines on a typical Android phone.
- Pinch with two fingers to zoom the text between 16 px and 64 px. Handle this with pointer events on the lyrics container, which has `touch-action: pan-y` so the browser scrolls but does not zoom the page. While pinching, keep the line under the fingers in place by adjusting `scrollTop` in proportion.
- Save the chosen size in `localStorage` as `kk.lyricsSize`. It applies to every round and every game. Settings has "Reset lyrics size".
- Save the lyrics scroll position with the game state, so a resumed round shows the same place.
- During the 3-2-1 countdown, the lyrics are already visible at the top, so the singer can read the first lines.

10.3 Scrub bar

- The track is 8 px high, spanning the width minus 16 px gutters. The touch target is 48 px high. The thumb is a 24 px circle.
- The played part is the neon pink accent. The unplayed part is dark grey.

- Instrumental stretches longer than 15 seconds (section 8.5) are drawn as amber segments over both parts of the track. Add a subtle diagonal stripe so they can be told apart without relying on colour.
- When a song has any stretches, show a tiny legend under the bar: an amber swatch and "no singing".
- Tap anywhere on the bar to jump there. Drag the thumb to scrub, with a bubble above the finger showing the target time. Seek only when the finger lifts. Ignore position updates while dragging, so the thumb does not jump back.
- "−10s" and "+10s" seek 10 seconds back or forward, clamped to the song.
- When the position is inside an instrumental stretch with more than 5 seconds of it left, show a pill above the bar: "Skip intro" if it is the first stretch starting at 0, otherwise "Skip instrumental". Tapping it seeks to 1 second before the stretch ends.
- Times use `m:ss`. The right label shows the total length.

10.4 Controls

Control	Behaviour
Play/pause	Toggles playback. While paused, show "Paused" faintly over the bar.
−10s / +10s	Seek, as above
Hold to skip	Must be held for 700 ms. A ring fills around the button while held. Releasing early cancels it and shows the toast "Hold to skip". On completion: a short vibration, then the skip (section 11).
Got It!	The most prominent button after play. One tap ends the round and opens the reveal (section 11).
Menu	Pauses nothing by itself. Choosing End game uses the confirmation from section 7.

10.5 Countdown and start-up

1. "I'm ready" unlocks audio (section 9), opens the round screen and shows a dark translucent overlay with 3, 2, 1.
2. Each number scales in and lasts 1 second. Android vibrates briefly on each.

3. Controls are disabled during the countdown.
4. At the end, the overlay fades and `playTrack` is called.
5. Until the first playing state arrives, the play button shows a spinner and the text "Starting…". After 8 seconds without audio, show "Spotify is taking a while" with a Retry button.

10.6 Orientation

The manifest locks portrait. If the phone is held in landscape anyway, cover the screen with "Turn your phone upright" and keep the music playing.

11. Ending a round and the game

A round ends in one of three ways: "Got It!", a held skip, or the song finishing. Only "Got It!" and a finished song lead to the reveal. Every path saves the game before changing screen.

11.1 Got It!

1. Remember whether the music was playing, then pause it and store the current position.
2. Fire a 1.5-second confetti burst over the screen. On Android, vibrate with the pattern `[30, 40, 60]`.
3. Open the Reveal screen with the answer card covered.

11.2 Reveal screen

- The card starts covered, showing "Hold the phone up and tap to reveal". This lets the singer turn the phone to the room first.
- Tapping the card flips it (a 400 ms 3D flip) to show:
  - a small label "It was…"
  - the played song's album art, square, `min(70vw, 320px)`, with a neon glow border
  - the played song's title in large bold text
  - the artists, joined with commas
  - a thin divider, then small muted text: "You were singing: <lyrics song title> by <artist>"
- Buttons under the card:
  - **Next singer** (primary). On the last round it reads **Finish**.

    - **Go back** (secondary), for an accidental "Got It!".
- If album art fails to load, show a neon music-note placeholder.

11.3 Go back

- Returns to the round screen exactly as it was: same lyrics scroll position, same text size, same playback position.
- If the music was playing when "Got It!" was pressed, resume it automatically. If it was paused or the song had ended, stay paused.
- The round is not counted as finished.

11.4 Next singer and Finish

- "Next singer" moves to the next round, saves, and shows Pass the phone for it.
- "Finish" (last round only) shows Game over.

11.5 Skipping a song

- After the 700 ms hold completes, pause the music and show the toast "Skipped".
- Move straight to the next round's Pass the phone screen. There is no reveal for a skipped song.
- If it was the last round, show Game over.
- The skipped song's tune and the lyrics it was paired with are simply not used. The pairing of later rounds does not change.

11.6 When the song finishes

When the engine emits `ended` (section 9.5):

1. Make sure Spotify is paused.
2. Show an overlay on the round screen: "That's the end of the song."
3. Buttons: **Reveal** (primary, opens the Reveal screen) and **Play it again** (seeks to 0 and plays).
4. The scrub bar stays usable. Tapping or dragging it closes the overlay and plays from the chosen point, for example to sing the chorus again.
5. "Got It!" also stays available.

11.7 Game over screen

- Large neon "Game over".
- Under it: "You played all 24 songs from <playlist name>." If the game was ended early: "You played 12 of 24 songs from <playlist name>." Count played rounds, including skipped ones.
- **Play again (new shuffle)**: re-runs preparation for the same playlist. The lyrics cache makes this quick. It makes a new shuffle, so the order and pairs differ.
- **Pick another playlist**: opens the picker.
- Delete the saved game as soon as this screen appears.
- No recap of songs, as Carlos asked.

11.8 Round state machine

`game.js` tracks one phase at a time. Screens only render the phase and call actions.

Phase	Entered by	Leaves to
`pass`	Start, Next singer, Skip	`countdown` (I'm ready), `over` (End game)
`countdown`	I'm ready	`playing` (after 3 s), `pass` (back button)
`playing`	Countdown end, Resume, Go back	`paused`, `ended`, `reveal`, `pass` (Skip), `over` (End game)
`paused`	Pause, menu, back button, interruption	`playing`, `reveal`, `pass` (Skip), `over` (End game)
`ended`	Engine `ended` event	`playing` (Play it again, or scrub), `reveal`
`reveal`	Got It!, Reveal	`pass` (Next singer), `over` (Finish), `playing` or `paused` (Go back)
`over`	Finish, End game	`preparing` (Play again), picker

12. Persistence, resume, wake lock and settings

A game in progress survives the app closing, the phone locking, or a reload. Small settings live in `localStorage`. Lyrics and the saved game live in IndexedDB.

12.1 Storage keys

Where	Key	Holds
localStorage	`kk.auth`	Access token, refresh token, `expires_at`, `authorized_at`, user id, display name
localStorage	`kk.pkce`	Verifier, state and Client ID during a login only
localStorage	`kk.clientId`	Custom Client ID, if set
localStorage	`kk.engine`	Remembered fallback choice, `'connect'`, if Automatic switched
localStorage	`kk.playbackMode`	`'auto'`, `'sdk'` or `'connect'`
localStorage	`kk.lyricsSize`	Lyrics font size in px
localStorage	`kk.vibration`	`true` or `false`
localStorage	`kk.seenHowTo`	`true` after the how-to was shown
IndexedDB `karaokry`, store `kv`	`lyrics:<track id>`	Cleaned lyrics, parsed synced lines, instrumental stretches, LRCLIB id, `fetchedAt`, or a "not found" marker
IndexedDB `karaokry`, store `kv`	`game:current`	The saved game (12.2)

Wrap every storage read and write in `try/catch`. If storage fails, the app still works for the current session.

12.2 Saved game shape

```
{
  "schema": 1,
  "savedAt": "2026-10-28T23:41:07Z",
  "userId": "spotify-user-id",
  "clientId": "client id used",
  "playlist": { "id": "...", "name": "Party Bangers", "imageUrl": "..." },
  "songs": [
    {
      "id": "...", "uri": "spotify:track:...", "name": "...", "artists":
```

```
["..."],
      "album": "...", "imageUrl": "...", "imageUrlSmall": "...",
"durationMs": 215000,
      "lyrics": "cleaned plain lyrics text",
      "instrumentalStretches": [{ "start": 0, "end": 18.4 }]
    }
  ],
  "round": 6,
  "phase": "playing",
  "positionMs": 73500,
  "wasPlaying": true,
  "lyricsScrollRatio": 0.42,
  "roundsPlayed": 6
}
```

`songs` is the locked order. The lyrics for round `i` are read from `songs[(i - 1 + N) % N].lyrics`.

Save the game:

- right after the shuffle
- on every phase change
- every 5 seconds while a song is playing
- on `visibilitychange` to hidden, and on `pagehide`

12.3 Resuming

- On launch, if `game:current` exists and its `userId` matches the logged-in user, show the Resume prompt (screen 4). If it belongs to another user, delete it.
- Phase `pass` or `countdown`: resume on Pass the phone for the saved round.
- Phase `playing`, `paused` or `ended`: open the round screen paused at the saved position, with the lyrics scroll restored. The play tap acts as the audio unlock, then calls `playTrack(uri, positionMs)`.
- Phase `reveal`: open the Reveal screen with the card covered.
- The SDK player is created and connected on the "Resume" tap, since a new page load has no player yet.
- If the saved `clientId` differs from the current one, still resume. Track URIs do not depend on the Client ID.

12.4 Keeping the screen awake

- Request `navigator.wakeLock.request('screen')` when a game starts or resumes.
- Re-request it on `visibilitychange` to visible, because Android releases it when the app is hidden.
- Release it on Game over and when leaving the game.
- If the API is missing or refuses, carry on silently.

12.5 Settings screen

Group	Item	Detail
Account	Logged in as <display name>	Read-only
Account	Log out	Section 6
Lyrics	Reset lyrics size	Shows the current size. Resets to the default.
Playback	Automatic / Play inside KaraoKry / Use the Spotify app	Section 9.2
Feedback	Vibration	On by default. Android only.
Help	How to play	Opens screen 3
Advanced (collapsed)	Use my own Spotify app (Client ID)	Sections 5 and 6
Advanced (collapsed)	Clear lyrics cache	Shows how many songs are cached. Confirms first.
Advanced (collapsed)	Debug info	Engine in use, device id, app version, last error message. A "Copy" button copies it as text.
About	Version and project link	From `config.js`

12.6 Install prompt and updates

- Capture Chrome's `beforeinstallprompt` event. Show an "Install KaraoKry" button on the Welcome and Playlist picker screens until the `appinstalled` event fires or the user dismisses it.

- The service worker caches the app shell under a versioned cache name. Requests to `spotify.com`, `scdn.co` and `lrclib.net` always go to the network and are never cached by the service worker.
- When a new service worker is waiting, show "New version ready. Tap to refresh." Only show it on the picker, Game over or Settings screens, never during a round.

13. Visual design

KaraoKry looks like a dark karaoke bar lit by neon signs: near-black purple backgrounds, hot pink and electric cyan glows, and big, friendly type. Carlos said standard fonts are fine, so it uses the phone's system font. The dark theme is the only theme, which suits dim party rooms.

13.1 Colour tokens

Define these as CSS custom properties on `:root` and use nothing else.

Token	Value	Used for
`--bg`	`#0A0612`	Page background
`--bg-raised`	`#140C22`	Cards, sheets, the lyrics panel
`--bg-raised-2`	`#1E1330`	Inputs, unplayed part of the scrub bar
`--line`	`#2E2245`	Dividers and borders
`--text`	`#F5F0FF`	Main text and lyrics
`--text-muted`	`#A99BC4`	Secondary text, the TUNE line, hints
`--pink`	`#FF2E97`	Primary accent: I'm ready, Got It!, played part of the bar, SING label
`--cyan`	`#22E4FF`	Secondary accent: play/pause ring, TUNE label, links
`--violet`	`#9B5CFF`	Background glow, confetti
`--amber`	`#FFB020`	Instrumental stretches on the bar, their legend
`--danger`	`#FF4D6D`	Errors and the End game confirmation

Glow recipe for neon elements: `box-shadow: 0 0 10px <colour>, 0 0 28px <colour at 40% opacity>`. Neon text uses three stacked `text-shadow` layers in the same colour at 4 px, 12 px and 28 px blur.

The page background adds two very faint radial "spotlights": pink at the top-left and violet at the bottom-right, each at about 12% opacity. The Pass the phone screen drops them and stays plain.

13.2 Type

- Font stack: `system-ui, -apple-system, "Segoe UI", Roboto, "Helvetica Neue", Arial, sans-serif`. No web fonts.
- Sizes: screen titles 28 px bold; body 17 px; small and labels 14 px; buttons 18 px bold.
- Display moments: "Game over" 44 px bold neon pink; countdown numbers 160 px bold neon cyan; songs-ready count 56 px bold.
- The wordmark "KaraoKry" is bold, slightly letter-spaced, with "Karao" in neon pink and "Kry" in neon cyan.
- Lyrics: default `clamp(22px, 6.5vw, 34px)`, weight 600, line height 1.35 (section 10.2).

13.3 Shapes and spacing

- Spacing on an 8 px grid. Screen side gutters 16 px.
- Cards and sheets: 16 px corner radius. Buttons and chips: fully rounded pills.
- Every tappable target is at least 48 by 48 px. Primary buttons are 56 px tall and full width, minus gutters.
- Play/pause is a 72 px circle with a cyan neon ring.
- "Got It!" is a pink filled pill with glow. "Hold to skip" is a muted outline pill.

13.4 Motion and feedback

- Screen changes: 200 ms fade with an 8 px upward slide.
- Countdown: each number scales from 0.6 to 1 and fades out over its second.
- Reveal card: 400 ms 3D flip.
- Confetti: a full-screen canvas with about 120 pieces in pink, cyan, violet and amber, falling for 1.5 seconds. It never blocks taps.
- Vibration (Android, when enabled): countdown tick 20 ms; Got It! `[30, 40, 60]`; skip completed 40 ms.

- With `prefers-reduced-motion`, replace the flip and slides with plain fades and use about 30 confetti pieces.
- No sound effects, because any sound would go into the singer's headphones.

13.5 Icon and manifest

- Icon: a microphone drawn as a neon-pink glowing tube outline, with a single cyan teardrop falling from it (the "cry" in KaraoKry), on a `--bg` rounded square.
- Provide `icons/icon.svg` as the master, plus `icon-192.png`, `icon-512.png` and `icon-maskable-512.png`. The maskable version keeps the artwork inside the central 80% safe zone with the background filling the full square.
- `manifest.webmanifest`: `name` and `short_name` "KaraoKry", `start_url` and `scope` `"./"`, `display` `"standalone"`, `orientation` `"portrait"`, `background_color` and `theme_color` `#0A0612`, and the icons above with `purpose` set correctly.

13.6 Voice and accessibility

- Copy is short, warm and a little cheeky, for example "Sing it wrong." and "Hiding the answers...". Every user-facing string in this document is final copy.
- Body text contrast is at least 4.5:1 against its background.
- Meaning never relies on colour alone: instrumental stretches also have stripes, and buttons have text labels.
- Every icon-only button has an `aria-label`. Focus outlines are visible.

14. Errors and edge cases

Every failure keeps the game state safe and tells the user one plain thing to do next. Never show raw error text or status codes outside Settings, Debug info.

Situation	How it is detected	What KaraoKry does
No internet at launch	`navigator.onLine` is false, or the first fetch fails	Full-screen message: "You're offline. KaraoKry needs the internet for Spotify and lyrics." Retry button.

Situation	How it is detected	What KaraoKry does
Connection drops mid-round	Fetch failures or SDK `not_ready`	Toast "Connection lost, trying again...". Keep the round. Retry play when the user taps play.
Spotify returns 5xx	HTTP status	Retry once after 1 s, then toast "Spotify had a hiccup. Try again."
Spotify returns 429	HTTP status, `Retry-After` header	Wait and retry, up to 3 times (section 9.6).
Login expired (6 months)	`invalid_grant` on refresh	Log in again, then offer to resume (section 6).
Account not on the guest list	403 from `GET /me`	Not on the guest list screen (section 6).
No Premium	SDK `account_error`, or 403 `PREMIUM_REQUIRED`	"Spotify Premium is needed to host a game on this phone."
Playlist unreadable	403 or no items field	Message from section 8.2.
Fewer than 3 usable songs	After filtering	Message from section 8.6.
Lyrics service down	5 network failures in a row	Message from section 8.4. Cached lyrics still work next time.
A song won't play mid-game	Play call fails with 403 or 404 for the track, or SDK `playback_error` on that track	Toast "This song won't play right now. Skipping it." Then behave as a skip (section 11.5).
Spotify autoplays another song	Track changes after the end (section 9.5)	Pause at once. Show the end-of-song overlay.
Someone takes over playback	Track or device changes before the end	Pause, then "Playback was interrupted. Tap play to continue."

Situation	How it is detected	What KaraoKry does
App sent to background, or a phone call	`visibilitychange`	Save. On return, if audio stopped, show "Tap play to continue".
App closed or killed	Next launch finds `game:current`	Resume prompt (section 12.3).
Two KaraoKry windows open	A `BroadcastChannel('karaokry')` message from the newer window	The older window disconnects its player and shows "KaraoKry is open in another window."
SDK script blocked or fails to load	Script `error` event or 10 s timeout	Treated as an SDK engine failure (section 9.2).
Headphones disconnect mid-song	Not reliably detectable in a browser	Android may move the music to the phone speaker. Nothing to build; the singer pauses. Mention it in the how-to fine print.
Album art missing or slow	Image `error` event	Neon music-note placeholder.
Very long titles	Layout	One line with an ellipsis on the round screen; wrap to two lines on the reveal.
Right-to-left or non-Latin lyrics	Content	Give each lyric line `dir="auto"`. Show explicit and non-English lyrics unchanged.
Storage full or blocked	Write throws	Skip caching. The current session continues.
Same song twice under different ids	Duplicate filter (section 8.3)	Kept once, so a song is never paired with itself.

15. Build phases, acceptance criteria and test plan

Build in two phases. Phase 1 is a rough but complete playable version for Carlos to test at a real party. Phase 2 adds resilience and polish. Carlos did not choose a build approach, so this is the default.

15.1 Phase 1: playable

1. Project skeleton, `config.js`, router, design tokens, manifest, icons, service worker, `README.md` with section 5's setup steps.
2. PKCE login, token refresh, guest-list screen, logout.
3. Playlist picker with search, usable and non-usable sections, and the copy tip.
4. Preparation: reading items, filters, LRCLIB lookup with cache, cleaning, instrumental stretches, shuffle, songs-ready screen.
5. SDK playback engine.
6. Pass the phone, countdown, round screen (lyrics scroll, pinch zoom, scrub bar with shading, ±10s, play/pause, hold to skip, Got It!).
7. Reveal with Go back, end-of-song overlay, Game over, Play again.
8. Saved game and resume, wake lock.
9. Deploy to GitHub Pages and test on Carlos's Android phone.

15.2 Phase 2: polish and resilience

1. Connect fallback engine and the Playback setting.
2. Skip-instrumental pill, confetti, vibration, reduced-motion handling.
3. How to play, full Settings, Debug info.
4. Android back-button table, install button, update toast.
5. Two-window guard, offline screen, every row of section 14.

15.3 Acceptance checklist

- [ ] Installs from Chrome on Android and opens full-screen, portrait, with the KaraoKry icon.
- [ ] Login works from the installed app and survives closing it. Tokens refresh after an hour with no prompt.
- [ ] An account not in User Management sees the guest-list screen, not a crash.

- [ ] The picker lists every playlist. Owned and collaborative ones work. Followed ones are greyed out with the copy tip. Search filters as you type.
- [ ] Local files, podcast episodes, tracks under 60 s, duplicates and songs without lyrics never appear in a game.
- [ ] The preparing screen shows only counts, never titles.
- [ ] With 5 songs locked as A to E, rounds pair as A/E, B/A, C/B, D/C, E/D (tune/lyrics).
- [ ] Nothing ever shows an upcoming song. Pass the phone shows only "Song x of N".
- [ ] "I'm ready" leads to 3-2-1, then the song plays in the connected headphones.
- [ ] Lyrics only move when the singer scrolls. Pinch zoom works and the size is remembered.
- [ ] Tapping or dragging the scrub bar lands within 1 second of the target. ±10s works at both ends of the song.
- [ ] Instrumental stretches over 15 s show in amber with stripes on songs that have synced lyrics.
- [ ] Skip only happens after a full 700 ms hold.
- [ ] "Got It!" pauses, shows confetti, and opens a covered card. A tap reveals art, title, artist and the lyrics song.
- [ ] "Go back" returns to the same lyrics position and resumes if the song was playing.
- [ ] When a song ends, Spotify does not carry on into another song, and the end-of-song overlay appears.
- [ ] The last round's reveal says "Finish" and leads to Game over. Play again gives a new order.
- [ ] Force-closing the app mid-song and reopening offers Resume, at the same song within 5 s of the position.
- [ ] The screen does not sleep during a 5-minute song.
- [ ] Forcing the Connect engine in Settings still plays, pauses, seeks and detects song ends.

15.4 Automated tests

Write plain-JavaScript tests runnable with `node --test`, for the logic modules that do not touch the DOM:

- Pairing: for N from 3 to 10, round 0 shows song N−1 and every other round shows song i−1.

- Shuffle: the output is a permutation of the input. Over 10,000 shuffles of 4 items, each position's frequency is within 5% of even.
- Filters: run fixture playlist responses in both the new (`items` / `item`) and old (`tracks` / `track`) shapes.
- Title cleaning: "Song - Remastered 2011", "Song (feat. X)", "Song - Live at Wembley" all clean to "song".
- Lyrics cleaning: section tags removed, blank-line runs collapsed, plain text rebuilt from synced lyrics when needed.
- Instrumental stretches: a fixture LRC with a 20 s intro, a 9 s gap and a 25 s solo yields exactly the intro and the solo.

15.5 Manual test on the phone

Ask Carlos to make a playlist named "KaraoKry test" containing: three well-known pop songs, one song with a long intro, one instrumental track, one podcast episode, the same song added twice, a song under 60 seconds, and one very obscure song. Then run a full game with Bluetooth headphones and work through the checklist in 15.3.

16. Assumptions and open questions

Where Carlos's answers left room for interpretation, this spec made the choices below. Build them as written unless Carlos changes them. Each is easy to reverse.

Topic	Assumption made	Easy alternative
"They can see both"	The singer sees the lyrics song's title and, in smaller muted text, the played song's title during the round.	Hide the played song's title until the reveal.
Confetti timing	Confetti fires on the "Got It!" tap. The card is then revealed by a second tap.	Fire confetti on the card flip instead.
Skip	A skipped song gets no reveal, and play moves straight to the next singer.	Show the reveal after a skip too.
Song finishing	The song ending shows an overlay with Reveal and Play it again. Carlos only mentioned "Got It!".	End silently and wait for "Got It!".

Topic	Assumption made	Easy alternative
Extra seek helpers	±10s buttons and a "Skip instrumental" pill were added to make jumping around easy, as Carlos asked.	Remove them and keep only the scrub bar.
Followed playlists	Shown greyed out with an explanation, since Carlos asked to see all playlists.	Hide them entirely.
Build approach	Two phases (section 15). Carlos did not answer this question.	Build everything in one pass.
Friend accounts	Up to 5 accounts in total on Carlos's Client ID, including his own if he logs in.	Friends use their own Client ID (section 5).
Platform	Android Chrome only for v1.	iPhone support later, which needs Safari-specific audio unlock testing.

Known risks

- **Lyrics accuracy.** LRCLIB may occasionally return lyrics for a different version or a different song with the same name. Hold to skip covers this.
- **Instrumental shading accuracy.** It is estimated from timed lyrics made for a possibly slightly different version of the track. Treat it as a guide, not an exact map.
- **Spotify policy changes.** Spotify changed its developer rules several times in 2026. Keep all Spotify calls inside `spotify-api.js` and the engines, so future changes touch few files.
- **Lyrics rights.** KaraoKry shows lyrics from a community database for private use among friends. It should not be published or promoted beyond that.

Questions to confirm with Carlos later (none block the build)

- Should a future version add scoring, teams or a timer? The code is structured so `game.js` can grow these.
- When should iPhone support come?

17. API reference and sources

These are the only outside calls KaraoKry makes. The Web API base URL is `https://api.spotify.com/v1`.

Service	Call	Used for
Spotify Accounts	`GET https://accounts.spotify.com/authorize`	Login (PKCE)
Spotify Accounts	`POST https://accounts.spotify.com/api/token`	Code exchange and token refresh
Web API	`GET /me`	User id, display name, guest-list check
Web API	`GET /me/playlists?limit=50`	Playlist picker, paged with `next`
Web API	`GET /playlists/{id}/items?limit=50&additional_types=track`	Playlist songs (owned or collaborative only), paged with `next`
Web API	`PUT /me/player/play?device_id={id}`	Start a track with `{ uris, position_ms }`, or resume with no body
Web API	`PUT /me/player/pause`	Pause (Connect engine)
Web API	`PUT /me/player/seek?position_ms={ms}`	Seek (Connect engine)
Web API	`PUT /me/player/repeat?state=off` and `PUT /me/player/shuffle?state=false`	Stop Spotify repeating or shuffling
Web API	`GET /me/player`	Playback position (Connect engine)
Web API	`GET /me/player/devices`	Find the phone's Spotify app (Connect engine)

Service	Call	Used for
Web API	`PUT /me/player` with `{ device_ids, play: false }`	Transfer playback to the phone (Connect engine)
Web Playback SDK	Script `https://sdk.scdn.co/spotify-player.js`; `Spotify.Player` methods `connect`, `disconnect`, `activateElement`, `pause`, `resume`, `seek`, `getCurrentState`, `addListener`	In-app playback (SDK engine)
LRCLIB	`GET https://lrclib.net/api/get?track_name=&artist_name=&album_name=&duration=`	Exact lyrics match
LRCLIB	`GET https://lrclib.net/api/search?track_name=&artist_name=`	Fallback lyrics search

Do not use:

- `GET /playlists/{id}/tracks` (replaced by `/items`) or batch calls such as `GET /tracks?ids=` (removed in 2026).
- Audio features and audio analysis, which new apps have not been able to use since late 2024.
- The implicit grant login flow (deprecated).
- Spotify search for public playlists, which cannot lead to a playable game (section 3).

Sources (checked 28 October 2026)

- February 2026 Web API Dev Mode Changes, Migration Guide, Spotify
- Web API Changelog, February 2026, Spotify
- Web API Changelog, July 2026, Spotify
- Introducing refresh token expiration, Spotify, 18 June 2026
- Web Playback SDK overview, Spotify
- Web Playback SDK is out of beta (mobile browser support), Spotify
- LRCLIB API documentation
- Spotify puts the brakes on developer mode with stricter API rules, Techzine